

Mini motor de búsqueda

Proyecto 1 — Estructuras de Datos y Algoritmos

Parte 1

Este apartado busca crear un índice de palabras, con una complejidad $O(1)$ promedio en inserción y en búsqueda. Es decir, dado un corpus de documentos, responder a: ¿en qué documentos aparece la palabra X? La forma más simple de hacerlo sería recorrer todos los documentos cada vez que se realiza una consulta, lo que llevaría a una complejidad de $O(n)$. Como esto resulta ineficiente, se recurre a estructuras de datos que optimicen el funcionamiento.

Para esto empleamos un índice invertido, una estructura que para cada palabra importante (más de 4 letras) guarde la lista de documentos en las que aparece. Para cumplir con los requisitos, necesitamos una estructura que, además de ser un índice invertido, permita insertar nuevas palabras o nuevos documentos a una palabra y buscar una palabra para obtener su lista de documentos. Además, la complejidad de estas operaciones debe ser de $O(1)$ promedio, en definitiva, deben ser operaciones rápidas.

Por esto, se escoge la tabla hash como estructura que garantiza el cumplimiento de todos los objetivos. En nuestro lenguaje de código, C++, esto se traduce a `unordered_map<string, vector<int>>`. La tabla hash almacena los pares clave $\rightarrow$ valor, de forma que las claves se emplean para la búsqueda y el valor es aquello que recuperas o insertas. La palabra se contiene en `string` (clave) y la lista de documentos se almacena en el `vector<int>` (valor). A pesar de ser una buena opción, esta estructura no ofrece un control de duplicados.

Parte 2

En el segundo apartado de este trabajo, proponen buscar una solución al caso en el que existan documentos duplicados en el corpus. En caso de que se implementara la parte 1 a un corpus con duplicados, aquel documento repetido aparecería dos veces en el índice y resultaría en salidas incorrectas. Aunque se podría añadir una comprobación manual de duplicados, esto

conllevar a aumentar el orden de complejidad a $O(n)$ para la inserción, por requerir un bucle.

Para evitar perjudicar el rendimiento, seleccionamos la misma estructura que antes, una tabla hash. Sin embargo, esta vez no se emplea un vector sino un `unordered_set`: `unordered_map<string, unordered_set<int>>`.

`unordered_set<int>` es también una tabla hash, pero sus elementos son a la vez clave y valor, y no admite duplicados. Manteniendo la complejidad de $O(1)$ promedio, se garantiza la unicidad de los elementos del índice de forma automática.

Parte 3

El tercer apartado propone consultar en qué documentos se encuentra una combinación de palabras o tokens dado un índice generado por la parte 1. El programa primero procesa el fichero de entrada `indice.txt` y lo convierte en una tabla hash que contiene el índice de palabras y documentos. Luego, lee el fichero con las consultas línea y convierte cada búsqueda en una instrucción postfix mediante el algoritmo Shunting Yard; esta notación ordena cada token de consulta (palabras y operadores) en un vector según el orden de operaciones preestablecido, en este caso priorizando la intersección AND sobre la unión OR. De la misma forma que la multiplicación precede a la suma, este orden de precedencia es por convención lógica y matemática. Utiliza una pila `std::stack` para guardar los operadores e introducirlos en la expresión postfix según la mayor precedencia. Luego, la instrucción es evaluada y su resultado es impreso en un fichero de salida.

Parte 4

Funcionalmente, el apartado 4 busca lo mismo que el apartado 3: consultar en qué documento se puede encontrar una combinación de palabras clave. Sin embargo, esta vez se consideran las consultas en un orden determinado por timestamps en el fichero de entrada `peticiones.txt`. En esta parte se utiliza de nuevo el índice generado en la parte 1 como entrada. Además, se define una estructura `Petition` para poder acceder al timestamp, usuario y la consulta en la cola de peticiones y al imprimir el fichero de salida.

Primero, se lee el fichero de peticiones y se crea una cola `std::queue` de peticiones para evaluar cada consulta según el orden de llegada. Mientras se procesa cada petición con el algoritmo Shunting Yard, se comprueba que la timestamp corresponda a un tiempo mayor al anterior, así verificando que la

cola accede siempre al elemento más antiguo. Una vez registradas todas las consultas, se imprime una confirmación de que el orden se ha respetado.

Funciones compartidas para las partes 3 y 4

En el enunciado, ambas partes tienen objetivos parecidos, simplemente con comportamientos de salida diferentes. Mientras que la parte 3 imprime directamente la consulta en postfix y el resultado de la búsqueda, la parte 4 requiere de alguna manera “guardar” las consultas ya procesadas para poder asegurarse de que se mantenga el orden FIFO en la cola. A pesar de que la estructura `queue` en sí garantiza el orden, resultó útil comprobar los timestamps en vez de simplemente imprimir “Orden respetado” en el fichero de salida. Por esta razón, antes de programar cada proceso por separado, se implementaron las siguientes funciones para uso dentro de cada apartado:

- `static std::unordered_map<std::string, std::vector<int>> cargarIndice(const std::string& archivo)`
 - Lee el archivo índice (generado en la parte 1) y devuelve una tabla hash con ids para cada palabra.
- `static bool esOperador(const std::string& t)`
 - Comprueba si una palabra es el operador AND, el operador OR, o una palabra a consultar (token)
- `static int precedencia(const std::string& op)`
 - Comprueba qué tipo de operador es y asigna un valor numérico para priorizar la intersección AND
- `static std::vector<std::string> infixAPostfix(const std::vector<std::string>& tokens)`
 - Algoritmo Shunting Yard para convertir una consulta infix a una instrucción postfix por tokens mediante una pila
- `static std::vector<int> interseccion(std::vector<int> a, std::vector<int> b)`
 - Define la intersección AND de dos vectores
- `static std::vector<int> unionVec(std::vector<int> a, std::vector<int> b)`
 - Define la unión OR de dos vectores
- `static std::vector<int> evaluarPostfix(const std::vector<std::string>& postfix, const std::unordered_map<std::string, std::vector<int>>& indice)`
 - Evalúa una consulta en formato postfix y devuelve el resultado de la búsqueda en el índice

- `static std::string formatearIds(const std::vector<int>& ids)`
 - Convierte un vector de ids a formato texto para impresión a ficheros.

Experimentos y análisis de rendimiento

Experimento 1: Tablas hash con y sin colisiones controladas

Indexa los mismos conjuntos de palabras y mide los tiempos para distintos tamaños de corpus:

Esta imagen representa la primera iteración de la creación del índice:

--- Experimento 1: tabla hash con distintos corpus ---		
Tamano	Insercion (us)	Busqueda (us)

1.000 palabras	4999	140
10.000 palabras	19764	49
100.000 palabras	109280	31

Esta imagen representa la segunda iteración de la creación del índice:

--- Experimento 1: tabla hash con distintos corpus ---		
Tamano	Insercion (us)	Busqueda (us)

1.000 palabras	3134	57
10.000 palabras	12491	36
100.000 palabras	53538	17

Es curioso ver que la primera vez que se ejecuta la creación de índice, el tiempo que toma es mucho mayor, esto no es por complejidad o errores en el primer documento. Esto ocurre porque el sistema operativo debe leer el archivo desde el disco físico y cargarlo en la memoria RAM y de ahí puede pasar a la caché del procesador, que es la más pequeña pero la más rápida.

Esta carga de información lleva tiempo, pero a partir de esa ejecución, si se repite la operación, lleva menos tiempo porque el SO ya ha guardado el archivo en el caché. Con `unordered_map` pasa lo mismo, la primera vez que se insertan los elementos, el sistema tiene que solicitar y mapear páginas de memoria nuevas. En ejecuciones posteriores esa memoria ya está disponible.

Es importante tener este fenómeno en cuenta al medir el rendimiento, hay que descartar la primera iteración, porque no es representativa.

Se puede observar que el tiempo de inserción de datos crece de forma aproximadamente de forma lineal, insertar cada palabra tiene $O(1)$ así que al insertar n palabras obtenemos $O(n)$: $3134 \rightarrow 12491 \rightarrow 53538$. Mientras que el tiempo de búsqueda se mantiene constante: $57 \rightarrow 36 \rightarrow 17$, confirmando $O(1)$. Teniendo en cuenta la segunda iteración, se puede ver que el orden de complejidad temporal se verifica de forma empírica.

Experimento 2: Tablas hash con y sin control de duplicados

Indexa un corpus con un 20% de documentos duplicados y compara:

- Tiempo de indexado con control de duplicados activo.
- Tiempo de indexado sin control (y verifica que los resultados son incorrectos).
- Tamaño del índice resultante en ambos casos.

Empleando un corpus de 100 documentos con 20 duplicados.

En esta imagen se representan los tiempo de inserción para un corpus con duplicados empleando estructuras con y sin control de duplicados:

--- Experimento 2: con y sin control de duplicados ---		
	Sin control (parte1)	Con control (parte2)
Tiempo insercion (us)	7630	11351
Entradas en indice	29	29
IDs totales en indice	1950	1147

IDs totales en índice representa el número total de valores guardados en `vector` o `unordered_set` en cada caso. Esto representa que si no hay control de duplicados, el número de valores aumenta, porque algunos aparecen más de una vez en la misma clave de forma errónea. Entradas índice cuenta el número de claves (palabras) que hay, y permite comprobar que efectivamente se crean índices para el mismo número de palabras en ambos casos.

Podría llamar la atención que el tiempo de inserción con control sea más largo. Sin embargo, las estructuras con control requieren calcular la función hash para `unordered_set` y es lógico que lleve más tiempo frente a el caso sin control, en que la operación de inserción es un simple `push_back()`. Se puede ver claramente que el índice obtenido mediante el primer método, no

es correcto. Los ficheros resultantes de este experimento sobrescribirán indice.txt (caso sin control) y indice_deduplicado.txt (caso con control de duplicados) que se hayan creado en otros apartados.

```
procesamiento -> [25, 42, 64, 64, 3, 3, 43, 28, 65, 65, 51, 5, 5, 18, 74, 74, 61, 29, 4, 68, 68, 4, 76, 76, 76, 26, 26, 26, 2, 30, 69, 67, 55, 74, 74, 11, 77, 67, 40, 56, 50, 13, 58, 14, 72, 45, 44, 19, 78, 44, 59, 59, 11]
inteligencia -> [25, 25, 15, 15, 28, 71, 6, 51, 1, 1, 1, 1, 18, 22, 4, 27, 4, 26, 26, 26, 6, 20, 20, 30, 30, 69, 69, 49, 70, 50, 7, 7, 32, 35, 22, 14, 72, 48, 17, 52, 52, 52, 45, 19, 78, 27, 60, 60, 57, 57, 57, 21, 59, 59, 52, 52, 52]
consulta -> [25, 25, 64, 64, 64, 64, 80, 3, 43, 28, 65, 65, 75, 75, 75, 51, 74, 75, 75, 75, 61, 61, 29, 27, 76, 46, 2, 2, 79, 30, 69, 67, 55, 37, 74, 67, 56, 56, 38, 58, 38, 72, 66, 66, 48, 48, 45, 41, 19, 78, 27, 57, 21, 8, 80, 10]
insercion -> [25, 80, 80, 15, 24, 24, 6, 34, 75, 51, 51, 12, 5, 16, 1, 18, 39, 39, 39, 75, 29, 4, 27, 68, 68, 34, 4, 23, 73, 46, 62, 26, 26, 6, 2, 2, 20, 20, 79, 33, 54, 49, 55, 55, 47, 11, 70, 63, 77, 40, 38, 38, 7, 73, 32, 32, 35, 16, 38, 38, 14, 72, 72, 48, 44, 41, 41, 41, 19, 78, 78, 44, 27, 62, 54, 57, 8, 8, 80, 80, 10, 39, 39, 39, 11]
```

Preguntas de análisis

1. ¿Por qué los motores de búsqueda usan tablas hash para los índices?

La tabla hash es la estructura perfecta porque identifica en qué documentos aparece la palabra clave de la forma más eficiente posible. El acceso es $O(1)$ promedio, así que da igual qué tan largo sea el vocabulario (cuántas claves haya).

¿En qué escenario cambiarías tu elección por otra estructura?

Si necesitáramos recorrer el vocabulario en orden alfabético — por ejemplo, para autocompletar consultas — cambiaríamos a un árbol rojo-negro `std::map`, que mantiene las claves ordenadas. El coste sería $O(\log n)$ por operación en lugar de $O(1)$, pero ganaríamos la capacidad de iterar en orden, algo que la tabla hash no puede garantizar.

2. ¿Qué ocurre si la función hash produce muchas colisiones?

Las colisiones ocurren cuando dos claves distintas, en este caso palabras, producen el mismo valor hash y se colocan en la misma posición de la tabla. La estructura empleada en C++ las gestiona con encadenamiento o direccionamiento abierto. Si hubiera muchas colisiones, la tabla de hash se convertiría en una lista enlazada, perdiendo los beneficios que nos aporta la función de hash.

¿Cómo afecta al rendimiento teórico y práctico?

En teoría, si la tabla de hash degenera en una lista enlazada, la complejidad promedio de $O(1)$ pasa a ser de $O(n)$. En la práctica se emplean funciones hash bien diseñadas para una distribución uniforme y mantener el factor de carga bajo (determina cuánto ha degenerado) realizando rehashing automático en cuanto sube un poco. Con un apropiado diseño de las funciones hash y un buen método de rehashing, el problema de las colisiones no es uno del que haya que preocuparse.

3. El operador AND equivale a una intersección de conjuntos y OR a una unión. ¿Cómo afecta el tamaño de los conjuntos intermedios al rendimiento del evaluador de consultas?

Ambos operadores necesitan recorrer completamente los conjuntos para buscar su unión o intersección, por lo que su complejidad es de $O(n + m)$, siendo n y m la longitud de cada vector. Sin embargo, el resultado de AND siempre es un vector igual o más pequeño que los vectores de entrada, mientras que el resultado de OR es un vector igual o más grande. Por convención, AND tiene precedencia sobre OR. Al evaluarse antes, los conjuntos intermedios se reducen, lo que reduce los elementos que OR debe recorrer y añadir a la unión, y hace que la operación sea más rápida.

4. ¿Qué pasaría si en lugar de una cola FIFO usaras una pila para gestionar las peticiones? ¿Qué usuarios se verían perjudicados?

Si se utilizara una pila para gestionar las peticiones, se procesarían en orden LIFO (Last In, First Out) en vez de en orden de llegada. Esto significa que la primera petición no estaría garantizada de ser procesada antes que las siguientes. Si siguen llegando peticiones, es posible que la primera petición quede “enterrada” al fondo de la pila y no se procese. Los usuarios más perjudicados serán los que lleguen primero, pues se los atenderá en última posición y tendrán mucho

tiempo de espera. En este caso, los usuarios 42 (la primera consulta, no la última) y 17 se verían más perjudicados.

5. Las bases de datos vectoriales modernas usan índices aproximados (HNSW, IVF) en lugar de búsqueda exacta. ¿Qué compromiso están haciendo? ¿Tiene sentido en tu implementación?

Los métodos modernos funcionan con una mayor cantidad de datos y a mayor velocidad que la búsqueda exacta, pero requieren más recursos en términos de memoria y procesamiento. Además, no tienen 100% de precisión como la búsqueda exacta. Para esta implementación, no tiene sentido utilizar estos métodos, ya que estamos trabajando con una escala mucho menor y buscamos que las palabras coincidan exactamente en el documento y en el índice.

Conclusión

El desarrollo de este mini motor de búsqueda ilustra uno de los principios fundamentales de la ingeniería de software: no existe una estructura ideal para todos los casos de aplicación. Cada componente del sistema ha requerido una decisión distinta por sus funcionalidades y objetivos de rendimiento.

Los experimentos han permitido verificar empíricamente las complejidades teóricas. Además, el experimento con duplicados ha demostrado que la elección de estructura puede no solo afectar al rendimiento, sino también a la corrección de los resultados.

En conclusión, el conocimiento de distintas estructuras, sus características y aplicaciones es necesario para poder tomar decisiones de diseño fundamentadas.